

授業グループ開発用 Firebase 導入・運用マニュアル

Vite / React + GitHub + Firebase を前提にした初心者向け手順書

作成日: 2026-06-09

この文書の位置づけ

本資料は、授業のグループ開発で Firebase を使うための作業手順書である。アカウント作成、メンバー招待、ローカル開発、Firestore、Hosting、GitHub 連携、トラブル対応を初心者向けにまとめる。

最初に読む結論

- **Firebase のアカウントは共有しない。**代表者が Firebase プロジェクトを作り、他メンバーを Google アカウントで招待する。
- **GitHub も共有アカウントにしない。**全員が自分の GitHub アカウントで作業し、ブランチとプルリクエストで統合する。
- **秘密情報を GitHub に置かない。**.env.local、サービスアカウント JSON、トークン類はコミットしない。
- **授業制作では機能を絞る。**基本は Firebase Hosting + Firestore + 必要なら Authentication までにする。
- **デプロイ担当は原則 1 人にする。**初心者チームでは、誰でも本番反映できる状態にすると事故が起きやすい。

目次

- 1. 前提と役割分担
- 2. Firebase で使う機能
- 3. アカウント・権限の作り方
- 4. 初回セットアップ
- 5. メンバー側の作業
- 6. Vite / React への組み込み
- 7. Firestore のデータ設計とルール
- 8. Firebase Hosting への公開
- 9. GitHub との連携
- 10. 担当者別の進め方
- 11. トラブルシュート
- 12. 発表・提出前チェックリスト
- 13. 参考資料・出典

1. 前提と役割分担

このマニュアルは、授業用の小規模 Web アプリを Vite / React + Firebase で作る前提で書いている。現在の main ブランチを基準に、各担当者が feature ブランチを切って作業し、Pull Request で統合する。

項目	前提
フロントエンド	Vite / React。コマンドは npm run dev, npm run build を想定する。
リポジトリ	GitHub。main に直接 push せず、feature ブランチから Pull Request を出す。
Firebase	代表者 1 人がプロジェクトを作成し、メンバーを個別 Google アカウントで招待する。
利用機能	Hosting, Firestore, 必要なら Authentication。Cloud Functions や Admin SDK は原則使わない。
料金プラン	無料枠で収まる範囲に限定する。課金設定が必要な機能は原則避ける。

初心者チーム向けの方針

Firebase は便利だが、設定箇所が多い。授業制作では「動く最小構成」を先に作り、Firestore の読み書きと Hosting 公開が確認できてから機能を増やす。

2. Firebase で使う機能

機能	使う目的	初心者向け判断
Firebase Hosting	作成した Web アプリを URL で公開する。	使う。発表時に URL を見せられる。
Cloud Firestore	投稿、設定、ユーザー情報などを保存する。	使う。小規模アプリのデータ保存に向く。
Firebase Authentication	ログイン状態やユーザー ID を扱う。	必要なら使う。匿名ログインが最も軽い。
Cloud Storage for Firebase	画像ファイルなどを保存する。	画像が必須の場合のみ検討。設定とルールが増える。
Cloud Functions	サーバ側処理を実行する。	今回は原則使わない。課金や難度が上がる。
Admin SDK	管理者権限で Firebase を操作する。	今回は使わない。秘密鍵管理が必要になる。

3. アカウント・権限の作り方

3.1 共有アカウントは禁止

Firebase は Google アカウントで管理する。1 つの Google アカウントの ID とパスワードを班全員で共有する運用は避ける。誰が変更したか追跡できず、削除・課金・設定ミスの責任も曖昧になる。

3.2 推奨ロール

人	Firebase 上の権限	理由
代表者	Owner	プロジェクト作成，メンバー招待，最終デプロイを担当する。
副代表者	Editor または Firebase Admin	代表者が不在でも最低限の作業を継続できる。
実装担当	Editor または Firebase Admin	Firestore や Hosting 設定を触る必要がある場合に付与する。
資料・発表担当	Viewer	画面確認だけなら閲覧権限で十分。

3.3 メンバー招待手順

1. 代表者が Firebase Console を開く。
2. 対象プロジェクトを開く。
3. 歯車アイコンから Project settings を開く。
4. Users and permissions を開く。
5. Add member からメンバーの Google アカウントを入力する。
6. ロールを選択して招待する。最初は Viewer でよい。実装で必要になったら Editor へ上げる。

権限のコツ

初心者チームでは、最初から全員 Owner にしない。設定を壊すリスクが大きい。Owner は代表者と副代表者までにする。

4. 初回セットアップ

4.1 代表者が Firebase プロジェクトを作る

1. Firebase Console で「プロジェクトを作成」を選ぶ。
2. プロジェクト名は授業・班名・年度が分かる名前にする。例: sdt2026-team-b-app
3. Google Analytics は必須ではない。授業制作では OFF でよい。

4. プロジェクト作成後、Web アプリを追加する。
5. アプリのニックネームを入力する。例: sdt-web
6. 表示される firebaseConfig を控える。後で .env.local に入れる。

4.2 Firestore を有効化する

1. Firebase Console 左メニューから Firestore Database を開く。
2. Create database を押す。
3. Production mode を選ぶ。後でルールを書く前提にする。
4. リージョンは asia-northeast1 など近い地域を選ぶ。授業用途なら一度決めたら変更しない。

4.3 Authentication を使う場合

1. Authentication を開く。
2. Sign-in method を開く。
3. 最小構成なら Anonymous を有効化する。
4. Google ログインを使う場合は Google を有効化し、サポートメールを選ぶ。

ログインを最小にする理由

発表用プロトタイプでは、メール認証やパスワード再設定まで作ると作業量が増える。ユーザー識別だけなら匿名ログインで十分な場合が多い。

5. メンバー側の作業

5.1 必要なもの

必要なもの	確認方法
Google アカウント	Firebase プロジェクトに招待されていること。
GitHub アカウント	リポジトリに Collaborator として招待されていること。
Node.js / npm	node -v と npm -v が表示されること。
VSCode	プロジェクトフォルダを開けること。
Firebase CLI	firebase --version が表示されること。デプロイ担当だけ必須。

5.2 ローカル環境確認コマンド

Windows PowerShell:

```
node -v
npm -v
git --version
```

Linux / macOS:

```
node -v
npm -v
git --version
```

5.3 リポジトリ取得と起動

Windows PowerShell:

```
git clone https://github.com/<owner>/<repo>.git
cd <repo>
npm install
npm run dev
```

Linux / macOS:

```
git clone https://github.com/<owner>/<repo>.git
cd <repo>
npm install
npm run dev
```

6. Vite / React への組み込み

6.1 Firebase SDK を入れる

Windows PowerShell:

```
npm install firebase
```

Linux / macOS:

```
npm install firebase
```

6.2 .env.example を作る

リポジトリには実値を入れない .env.example を置く。メンバーはこれを見て .env.local を作る。

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
```

6.3 .env.local を作る

.env.local は各自の PC だけに置く。GitHub にコミットしない。

```
VITE_FIREBASE_API_KEY=Firebase Console で表示された値
VITE_FIREBASE_AUTH_DOMAIN=xxxx.firebaseio.com
VITE_FIREBASE_PROJECT_ID=xxxx
```

```
VITE_FIREBASE_STORAGE_BUCKET=xxxx.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=xxxx
VITE_FIREBASE_APP_ID=xxxx
```

6.4 .gitignore を確認する

```
node_modules
dist
.env
.env.local
.env.*.local
firebase-debug.log
.firebasearc.local
serviceAccount*.json
```

6.5 Firebase 初期化ファイルを作る

例: src/lib/firebase.ts

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};

export const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

API key の扱い

Firebase の Web アプリ用 apiKey は通常の秘密鍵とは性質が異なる。ただし、Firestore Security Rules や Authentication 設定を正しく作ることが前提である。env.local に置くのは、値を隠すためというより、環境ごとの設定を分離するためである。

7. Firestore のデータ設計とルール

7.1 最小データモデル

掲示板・投稿型アプリを想定する場合の最小例。実際の画面名に合わせて collection 名は変更してよい。

collection	document 例	主なフィールド
posts	自動 ID	title, body, imageUrl, createdAt, uid, authorName

collection	document 例	主なフィールド
users	uid	displayName, createdAt
settings	global	demoMode, announcement

7.2 投稿追加の例

```
import { addDoc, collection, serverTimestamp } from "firebase/firestore";
import { db } from "../firebase";

export async function createPost(uid: string, title: string, body: string) {
  await addDoc(collection(db, "posts"), {
    uid,
    title,
    body,
    createdAt: serverTimestamp(),
  });
}
```

7.3 投稿取得の例

```
import { collection, getDocs, orderBy, query } from "firebase/firestore";
import { db } from "../firebase";

export async function fetchPosts() {
  const q = query(collection(db, "posts"), orderBy("createdAt", "desc"));
  const snap = await getDocs(q);
  return snap.docs.map((doc) => ({ id: doc.id, ...doc.data() }));
}
```

7.4 ルールの基本方針

- 発表デモだけでも、Firestore を完全公開にしたまま放置しない。
- 読み取りは公開，作成はログイン済みユーザーのみ，更新・削除は投稿者本人のみ，という形が最小限扱いやすい。
- Authentication を使わない場合は，本番公開前に書き込み権限を閉じる。

例: posts を使う場合の Firestore Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /posts/{postId} {
      allow read: if true;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
        && request.resource.data.title is string
        && request.resource.data.body is string;
      allow update, delete: if request.auth != null
        && request.auth.uid == resource.data.uid;
    }
  }
}
```

```
}  
}
```

一時的なデモ用ルール

allow read, write: if true; は動作確認には便利だが、公開したままにしない。提出・発表前には最低限の制限ルールに戻す。

8. Firebase Hosting への公開

8.1 CLI のインストール

デプロイ担当者だけでよい。

Windows PowerShell:

```
npm install -g firebase-tools  
firebase --version  
firebase login
```

Linux / macOS:

```
npm install -g firebase-tools  
firebase --version  
firebase login
```

8.2 Hosting 初期化

プロジェクトルートで実行する。

```
firebase init hosting
```

質問されたら以下を選ぶ。

質問	選択
Which Firebase project?	代表者が作成したプロジェクトを選ぶ。
What do you want to use as your public directory?	Vite なら dist
Configure as a single-page app?	React Router を使うなら Yes。使わなくても基本 Yes でよい。
Set up automatic builds and deploys with GitHub?	最初は No。手動デプロイで安定させる。
File dist/index.html already exists. Overwrite?	通常は No。

8.3 ビルドとデプロイ

Windows PowerShell:

```
npm run build  
firebase deploy --only hosting
```

Linux / macOS:


```
npm run build
firebase deploy --only hosting
```

デプロイ担当を 1 人にする

班員全員が本番デプロイすると、どの変更が反映されたか分からなくなる。main ブランチにマージ済みのものだけを代表者がデプロイする。

9. GitHub との連携

9.1 ブランチ運用

作業開始前に必ず main を最新化してからブランチを切る。

```
git checkout main
git pull origin main
git checkout -b feature/firebase-init
```

作業後はコミットして push する。

```
git status
git add .
git commit -m "feat: add Firebase initialization"
git push -u origin feature/firebase-init
```

9.2 GitHub Actions で最低限の CI を入れる

Firebase デプロイを自動化する前に、まず「ビルドできるか」だけを自動確認する。これだけでも授業で CI を使った説明ができる。

例: .github/workflows/ci.yml

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
```

9.3 GitHub Secrets に入れるもの

値	入れる場所	備考
Firebase Web config	通常は不要	Vite の.env.local で各自管理。ビルド時に必要なら GitHub Secrets に入れる。
サービスアカウント JSON	原則作らない	Admin SDK や自動デプロイで必要になり得るが、初心者制作では避ける。
Firebase token	非推奨寄り	運用が雑になりやすい。手動デプロイで十分。
GitHub 連携の範囲 授業制作では、GitHub Actions で build が通ることを確認し、デプロイは手動にする構成が安全。時間が余った場合だけ Firebase Hosting の GitHub 連携を検討する。		

10. 担当者別の進め方

担当	やること	完了条件
A: 代表者	Firebase プロジェクト作成，メンバー招待，.env.example 作成，Hosting 初期化，最終デプロイ。	firebaseConfig を班員に共有し，URL でアプリを表示できる。
B: データ担当	Firestore の collection 設計，投稿追加・取得関数，Rules の初稿作成。	ローカルでデータを追加・表示できる。
C: UI 連携担当	画面から B の関数を呼び出し，フォーム入力・一覧表示・エラー表示を作る。	画面操作だけで Firestore に反映される。
D: 動作確認・CI 担当	README 更新，npm run build 確認，GitHub Actions の build 設定，発表用チェックリスト作成。	Pull Request で CI が成功し，README 通りに起動できる。

10.1 B 担当の細かい手順

1. src/lib/firebase.ts が存在するか確認する。
2. src/features/posts/postRepository.ts など，Firestore 操作を 1 ファイルにまとめる。
3. createPost, fetchPosts など画面から呼ぶ関数を用意する。
4. Firestore Console で実際に document が作られるか確認する。
5. Rules を変更した場合は，何を許可して何を禁止したか README に書く。

10.2 C 担当の細かい手順

1. 画面側では Firestore の詳細を直接書かず，B 担当の関数を呼ぶ。
2. 送信中，成功，失敗の状態を画面に出す。
3. 入力が空の場合は送信しない。
4. npm run build でエラーが出ないことを確認してから Pull Request を出す。

10.3 D 担当の細かい手順

1. README に「環境構築」「.env.local の作り方」「起動方法」「ビルド方法」を書く。
2. GitHub Actions の build が Pull Request で動くようにする。
3. 代表者以外の PC でも README 通りに起動できるか確認する。
4. 発表前に Hosting の URL，GitHub リポジトリ，使用技術の説明を 1 枚にまとめる。

11. トラブルシューティング

症状	原因候補	確認・対処
Firebase: Error auth/Unauthorized-domain	Authentication でドメイン許可が不足。	Firebase Console > Authentication > Settings > Authorized domains を確認する。localhost は通常入っている。
Missing or insufficient permissions	Firestore Rules で拒否されている。	request.auth が必要なのに未ログイン、または uid 不一致。Rules とログイン状態を確認する。
npm run build が通らない	型エラー、import ミス、環境変数未定義。	ローカルで npm run build を実行し、最初のエラーから直す。
firebase deploy でプロジェクトが出ない	Firebase プロジェクト権限がない、または別アカウントで login 中。	firebase login --reauth を実行し、Firebase Console で権限を確認する。
デプロイ後に真っ白	Vite のビルド失敗、public directory 誤り、SPA 設定不足。	npm run build 後に dist ができているか、firebase.json の public が dist か確認する。
他人の PC で動かない	.env.local がない、npm install 未実行。	.env.example を見て .env.local を作り、npm install を実行する。

12. 発表・提出前チェックリスト

12.1 Firebase

- 共有 Google アカウントを使っていない。
- Firebase プロジェクトに必要なメンバーだけが招待されている。
- Firestore Rules が完全公開のままではない。
- 不要な課金機能を使っていない。
- Hosting URL でアプリが開ける。

12.2 GitHub

- .env.local やサービスアカウント JSON がコミットされていない。
- main ブランチで npm run build が成功する。
- Pull Request を使った履歴が残っている。
- GitHub Actions の build が成功している。
- README に起動方法と Firebase 設定方法が書かれている。

12.3 発表で説明するポイント

- Firebase Hosting で Web アプリを公開したこと。
- Cloud Firestore でデータ保存を行ったこと。

- GitHub のブランチと Pull Request で統合したこと。
- GitHub Actions でビルド確認を自動化したこと。
- Firestore Rules や .env 管理により，最低限のセキュリティを考慮したこと。

発表時の言い方

「Firebase を使いました」だけでは弱い。何を Firebase に任せたのか，どの設定を自分たちで行ったのか，GitHub や CI とどう連携したのかを説明すると，授業内容との接続が明確になる。

参考資料・出典

- Firebase Documentation, “Add Firebase to your JavaScript project”, accessed 2026-06-09.
<https://firebase.google.com/docs/web/setup>
- Firebase Documentation, “Manage project access with Firebase IAM”, accessed 2026-06-09.
<https://firebase.google.com/docs/projects/iam/overview>
- Firebase Documentation, “Owner, Editor, and Viewer roles”, accessed 2026-06-09.
<https://firebase.google.com/docs/projects/iam/roles-basic>
- Firebase Documentation, “Get started with Firebase Hosting”, accessed 2026-06-09.
<https://firebase.google.com/docs/hosting/quickstart>
- Firebase Documentation, “Firebase CLI reference”, accessed 2026-06-09. <https://firebase.google.com/docs/cli>
- Firebase Documentation, “Get started with Cloud Firestore Security Rules”, accessed 2026-06-09.
<https://firebase.google.com/docs/firestore/security/get-started>
- Firebase Documentation, “Security Rules and Firebase Authentication”, accessed 2026-06-09.
<https://firebase.google.com/docs/rules/rules-and-auth>
- Firebase Documentation, “Learn about using and managing API keys for Firebase”, accessed 2026-06-09.
<https://firebase.google.com/docs/projects/api-keys>
- GitHub, “GitHub Actions”, accessed 2026-06-09. <https://github.com/features/actions>
- ソフトウェア開発論 第 1 回資料: 演習発表と 2 回目～7 回目で扱った技術に関する開発演習の説明.
- ソフトウェア開発論 第 2 回・第 3 回資料: GitHub, ブランチ, マージ, プルリクエストの説明.
- ソフトウェア開発論 第 7 回資料: 継続的インテグレーション, ビルド, テスト, デプロイの説明.